7zpus.swe@gmail.com

Specifica Tecnica

Tabella di Versionamento

Versione	Data	Autore	Verificatore	Descrizione
0.1.0	2026/02/28	Laoud Zakaria	Georgescu Diana	Creazione del documento e stesura struttura iniziale

Indice

1 Introduzione **5**
1.1 Obiettivo del documento . . . 5
1.2 Glossario . . . 5
1.3 Riferimenti . . . 5
1.3.1 Riferimenti normativi . . . 5
1.3.2 Riferimenti informativi . . . 5

2 Tecnologie utilizzate **7**

3 Linguaggio di Programmazione: TypeScript **7**
3.1 Deployment: Electron . . . 7
3.2 Framework Frontend: Angular . . . 8
3.3 Librerie Aggiuntive . . . 9
3.4 Tecnologie di analisi statica . . . 10
3.5 Tecnologie di analisi dinamica . . . 10

4 Contesto del sistema **10**
4.1 Modello architetturale . . . 11
4.2 Descrizione degli attori esterni . . . 11

5 Architettura dei container **12**
5.1 Renderer . . . 13
5.1.1 Struttura architetturale . . . 13
5.2 [Main] . . . 13
5.2.1 Tecnologia: Node.js . . . 13

6 Architettura delle componenti **14**
6.1 [Componente 1] . . . 14
6.1.1 Descrizione . . . 14
6.1.2 Interfacce . . . 14
6.1.3 Dipendenze . . . 14
6.1.4 Tracciamento dei requisiti . . . 14
6.2 [Componente 2] . . . 14
6.2.1 Descrizione . . . 14
6.2.2 Interfacce . . . 14
6.2.3 Dipendenze . . . 14
6.2.4 Tracciamento dei requisiti . . . 14
6.3 Componenti di Ricerca (FrontEnd) . . . 15
6.3.1 Descrizione . . . 15

7 Backend – Search **15**
7.1 Panoramica . . . 15
7.2 Architettura del modulo . . . 16
7.2.1 SearchIpcAdapter . . . 17

7.2.2 Use Case di ricerca 18
7.2.3 Modelli di dominio per la ricerca 21
7.2.4 Mapper di ricerca 22
7.2.5 Repository e DAO di ricerca 25
7.2.6 DocumentMetadataQueryBuilder 27

8 Backend – Export e Stampa 28
8.1 Panoramica 28
8.2 Architettura del modulo 29
8.3 Componenti 30
8.3.1 FileViewerIpcAdapter 30
8.3.2 Use Case 31
8.3.3 ExportResult 31
8.3.4 LocalExportPort e PrintPort 32

Elenco delle figure

1 Diagramma C1 - Contesto del sistema *DIPReader* 11
2 Diagramma C2 - Container del sistema *DIPReader* 12
3 Diagramma C3 – Componenti del modulo di ricerca 16
4 Diagramma C3 – Componenti del modulo di ricerca (Repository e DAO) . 17
5 Diagramma delle classi – SearchIpcAdapter 17
6 Diagramma delle classi – SearchDocumentalClassUC 18
7 Diagramma delle classi – SearchProcessUC 19
8 Diagramma delle classi – SearchDocumentsUC 19
9 Diagramma delle classi – SearchSemanticUC 20
10 Diagramma delle classi – GetCustomMetadataKeysUC 20
11 Diagramma delle classi – Query Model: SearchDocumentsQuery, SearchGroup, SearchCondition 21
12 Diagramma delle classi – DocumentClassMapper 23
13 Diagramma delle classi – ProcessMapper 23
14 Diagramma delle classi – DocumentMapper 24
15 Diagramma delle classi – MetadataKeyMapper 24
16 Diagramma delle classi – DocumentClassRepository e DAO 25
17 Diagramma delle classi – ProcessRepository e DAO 25
18 Diagramma delle classi – DocumentRepository e DAO 26
19 Diagramma delle classi – DocumentMetadataQueryBuilder 27
20 Diagramma C3 – Componenti del modulo di export e stampa 29
21 Diagramma delle classi – FileViewerIpcAdapter, Use Case e ExportResult 30
22 Diagramma delle classi – Use Case e Ports 31
23 Diagramma delle classi – Infrastructure Layer: adapter e port 32

Elenco delle tabelle

2 Dettagli sulla scelta di TypeScript come linguaggio di programmazione . 7
4 Dettagli sulla scelta di Electron come piattaforma di deployment 7
6 Dettagli sulla scelta di Angular come framework frontend 8
8 Dettagli sulle scelte di librerie aggiuntive 9
10 Dettagli sulle scelte di tecnologie di analisi statica 10
12 Dettagli sulle scelte di tecnologie di analisi dinamica 10

1 Introduzione

1.1 Obiettivo del documento

L'obiettivo del presente documento è descrivere in modo rigoroso le scelte tecnologiche e l'architettura del prodotto *DIPReader*, adottando il modello di progettazione *C4*.
L'analisi verrà condotta partendo dalla descrizione del contesto applicativo, per poi approfondire la struttura dei container, delle componenti e, infine, del codice, al fine di fornire una rappresentazione completa, coerente e sistematica del sistema.

1.2 Glossario

Ogni termine tecnico o con un significato particolare nell'ambito dell'Ingegneria del Software, utilizzato nella documentazione di progetto, è definito nell'apposito documento Glossario (v2.0.0).

1.3 Riferimenti

1.3.1 Riferimenti normativi

- Norme di Progetto v2.0.0 (v2.0.0)
- Standard SDD/IEEE/1016:2009(ultimo accesso: 02/03/2026)
- Regolamento di Progetto Didattico a.a. 2025/2026 (ultimo accesso: 02/03/2026)

1.3.2 Riferimenti informativi

- Capitolato C3: DIPReader (ultimo accesso: 10/02/2026)
- Dispense su Dependency Injection (Ultimo accesso: 02/03/2026)
- Dispense su OOP (Ultimo accesso: 02/03/2026)
- Dispense su Diagrammi delle classi (Ultimo accesso: 02/03/2026)
- Dispense su Pattern architetturali (Ultimo accesso: 02/03/2026)
- Dispense su Pattern creazionali (Ultimo accesso: 02/03/2026)
- Dispense su Pattern strutturali (Ultimo accesso: 02/03/2026)
- Dispense su Principi SOLID (Ultimo accesso: 02/03/2026)
- Dispense sulla Progettazione software (Ultimo accesso: 02/03/2026)
- Dispense sulla Qualità del software (Ultimo accesso: 02/03/2026)
- Repository Architettura esagonale (Kotlin) (Ultimo accesso: 02/03/2026)

- Repository Architettura esagonale (Java) (Ultimo accesso: 02/03/2026)
- Repository Ingegneria del software - swe-imdb (Ultimo accesso: 02/03/2026)
- C4 Model (Ultimo accesso: 03/03/2026)
- PlantUML (Ultimo accesso: 03/03/2026)
- Riferimenti a scelte tecnologiche:
  - TypeScript (Ultimo accesso: 03/03/2026)
  - Angular (Ultimo accesso: 03/03/2026)
  - Electron (Ultimo accesso: 03/03/2026)
  - SQLite (Ultimo accesso: 03/03/2026)
  - sqlite-vss (Ultimo accesso: 03/03/2026)
  - ONNX Runtime (Ultimo accesso: 03/03/2026)
  - Transformers.js (Ultimo accesso: 03/03/2026)
  - Node.js (Ultimo accesso: 03/03/2026)
- Glossario (v2.0.0)
- Piano di Progetto (v2.0.0)
- Piano di Qualifica (v2.0.0)
- Verbali interni:
- Verbali esterni:

2 Tecnologie utilizzate

La scelta delle teccnologie è stata guidata dalla necessità di rispetta i vincoli richiesti dalla proponente legati ad autoconsistenza, portabilità e facilità d'uso, nonché da considerazioni relative alla facilità di sviluppo, alla disponibilità di risorse e alla compatibilità con i requisiti funzionali e non funzionali del sistema.

3 Linguaggio di Programmazione: TypeScript

Scelta	Versione	Motivazione
TypeScript	5.9.2	Scelto per la sua capacità di fornire un sistema di tipi statico che migliora la sicurezza del codice e facilita la manutenzione, riducendo il rischio di errori comuni durante lo sviluppo. Inoltre, TypeScript è ampiamente supportato da un ecosistema ricco di librerie e strumenti, e si integra perfettamente con tecnologie come Angular ed Electron.

Tabella 2: Dettagli sulla scelta di TypeScript come linguaggio di programmazione

3.1 Deployment: Electron

Scelta	Versione	Motivazione
Electron	30.0.0	Scelto per la sua capacità di consentire lo sviluppo di applicazioni desktop multipiattaforma utilizzando tecnologie web, garantendo così la portabilità del prodotto su diversi sistemi operativi. Electron permette di creare un'esperienza utente nativa, sfruttando le potenzialità di Node.js per l'accesso al filesystem e alle risorse locali, facilitando l'integrazione con librerie e strumenti di terze parti.

Tabella 4: Dettagli sulla scelta di Electron come piattaforma di deployment

3.2 Framework Frontend: Angular

Scelta	Versione	Motivazione
Angular	21.2.5	Scelto per la sua capacità di supportare lo sviluppo di applicazioni web complesse e scalabili, con un'architettura basata su componenti che favorisce la modularità e la manutenibilità del codice. Inoltre, Angular offre un'ampia gamma di strumenti e funzionalità integrate, come la gestione dello stato, il routing e il supporto per la creazione di interfacce utente reattive, che lo rendono adatto a soddisfare i requisiti funzionali del progetto. La sua implementazione della Dependency Injection facilita la gestione delle dipendenze tra i componenti, migliorando la testabilità e la manutenibilità del codice.

Tabella 6: Dettagli sulla scelta di Angular come framework frontend

3.3 Librerie Aggiuntive

Scelta	Versione	Motivazione
Better SQLite	12.8.0	SQLite è un database relazionale che non richiede un server separato e può essere integrato direttamente nell'applicazione, facilitando lo sviluppo e il deployment. La sua leggerezza e portabilità lo rendono adatto a un'ampia gamma di piattaforme, garantendo la compatibilità con i requisiti di portabilità del progetto.
sqlite-vss	0.1.2	Scelto per la sua capacità di fornire una versione virtuale di SQLite, che consente di eseguire query su dati in memoria senza la necessità di creare un database fisico dei vettori per la ricerca semantica, semplificando così l'integrazione con il sistema.
ONNX Runtime	1.17.3	Scelto per la sua capacità di eseguire modelli di machine learning in modo efficiente e flessibile, supportando una vasta gamma di modelli e framework, facilitando l'integrazione di funzionalità di intelligenza artificiale nel prodotto.
Transformers.js	3.8.1	Scelto per la sua capacità di fornire un'interfaccia JavaScript per l'utilizzo di modelli di trasformatori, facilitando l'integrazione di funzionalità avanzate di elaborazione del linguaggio naturale e machine learning nel prodotto.

Tabella 8: Dettagli sulle scelte di librerie aggiuntive

3.4 Tecnologie di analisi statica

Scelta	Versione	Motivazione
SonarQube	2026.2	Scelto per la sua capacità di identificare e segnalare pattern di codice problematici, migliorando la qualità del codice e facilitando la manutenzione del progetto. Integrato nella pipeline di sviluppo, SonarQube consente di monitorare costantemente la qualità del codice, identificando potenziali bug, vulnerabilità e problemi di manutenzione, contribuendo così a garantire un prodotto finale robusto e affidabile.
Prettier	2.9.3	Scelto per la sua capacità di formattare automaticamente il codice in modo coerente, migliorando la leggibilità e facilitando la collaborazione tra i membri del team. Prettier supporta una vasta gamma di linguaggi e può essere facilmente integrato con editor di codice e strumenti di sviluppo, semplificando il processo di mantenimento di uno stile di codifica uniforme.

Tabella 10: Dettagli sulle scelte di tecnologie di analisi statica

3.5 Tecnologie di analisi dinamica

Scelta	Versione	Motivazione
Vitest	2.3.1	Scelto per la sua capacità di fornire un ambiente di test per applicazioni JavaScript e per il suo controllo dei componenti di Mock.
Playwright	1.59.1	Scelto per la sua capacità di fornire un ambiente di test per applicazioni web, con possibilità di testare flussi completi di interazione utente. Utilizzato quindi per i test E2E.

Tabella 12: Dettagli sulle scelte di tecnologie di analisi dinamica

4 Contesto del sistema

I vincoli di progetto, come specificato nell'Analisi dei Requisiti, richiedono che il sistema sia portabile, facile da usare e autoconsistente. Il caso d'uso principale del sistema è quello di un utente che desidera esplorare e interrogare un DIP per estrarre documenti fiscali e non, potendone verificare la validità e autenticità.

4.1 Modello architetturale

I vincoli di progetto hanno guidato la progettazione del sistema, influenzando le scelte tecnologiche e architetturali. Il sistema progettato quindi è pensato per non utilizzare tecnologie che richiedano installazioni o configurazioni complesse da parte dell'utente. I servizi offerti sono tutti accessibili in assenza di connettività, complesa la ricerca semantica.
Gli unici attori esterni coinvolti sono il file *system locale* da cui viene letto il DIP. STAMPA DA VEDERE SE HA SENSO METTERLO

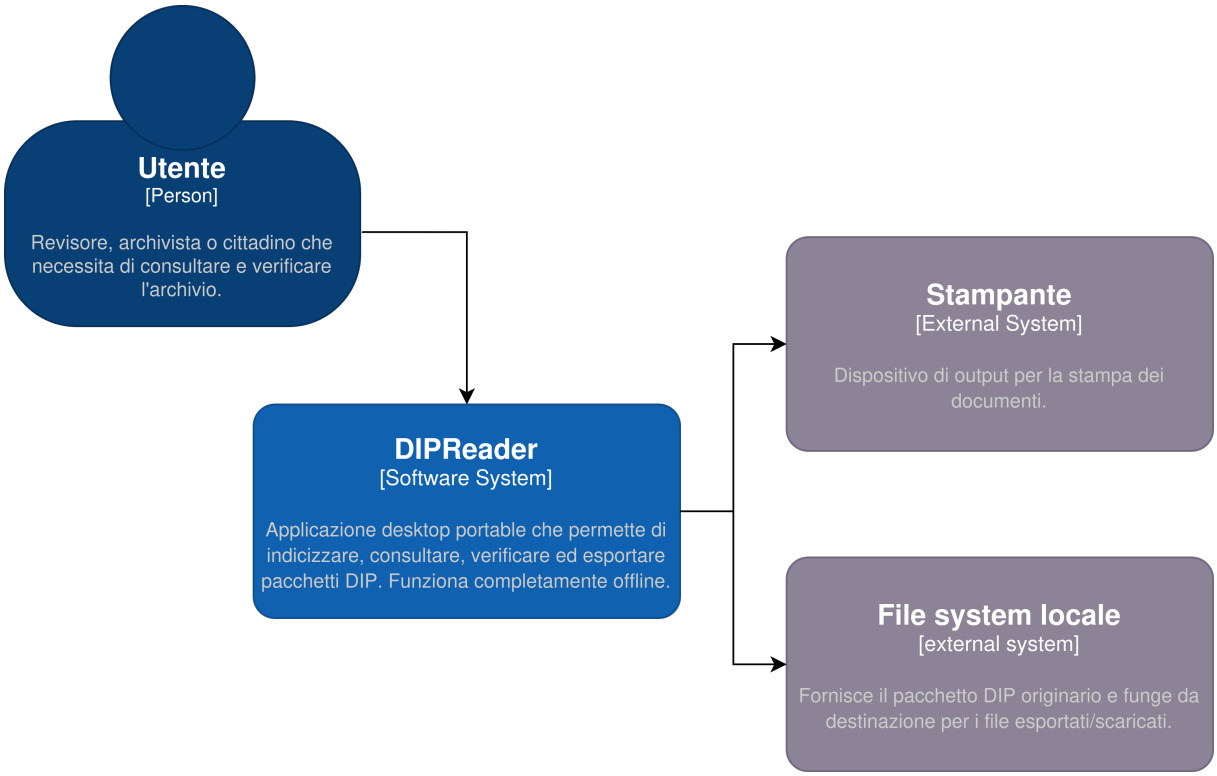

Figura 1: Diagramma C1 - Contesto del sistema *DIPReader*

4.2 Descrizione degli attori esterni

Gli unici attori esterni come già detto sono:

- il file system locale da cui viene letto il DIP
- STAMPA DA VEDERE SE HA SENSO METTERLO

5 Architettura dei container

La progettazione del sistema *DIPReader* è stata guidata dai principi di modularità, scalabilità e manutenibilità, con l'obiettivo di creare un'architettura che possa evolversi nel tempo e adattarsi a nuove esigenze e normative AGID.
In particolare, è stata completamente scorporata la logica di business dalla logica di presentazione, esponendo come unico punto di accesso comune gli *IPC channel*.
Il sistema così facendo permette di avere un backend facilmente estendibile, modificabile e soprattutto che si adatta allla struttura del DIP autonomamente. D'altra parte il frontend è completamente disaccoppiato ma strutturato in modo che i componenti di presentazione siano facilmente sostituibili, modificabili e soprattutto adattabili a nuove esigenze.

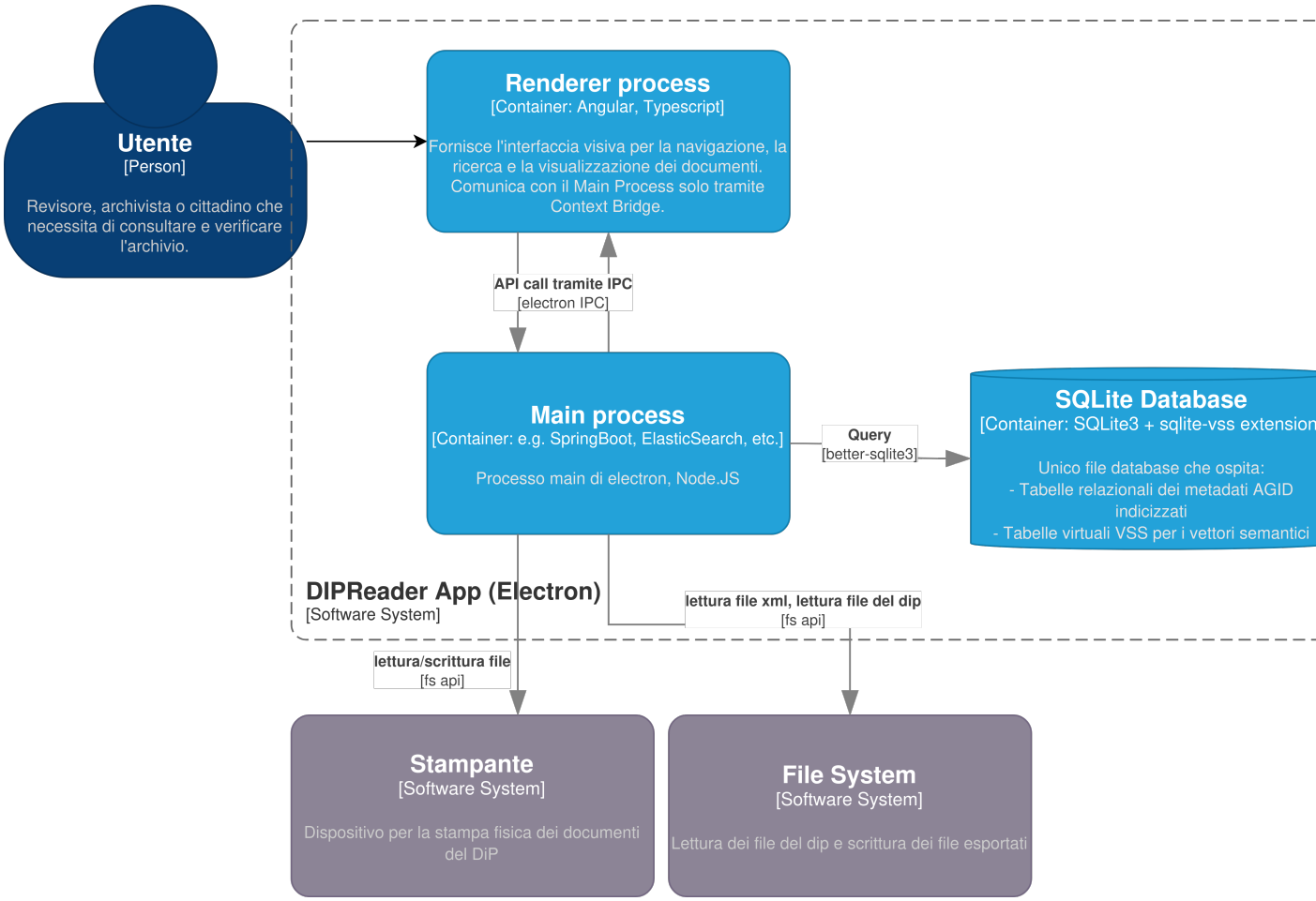

Figura 2: Diagramma C2 - Container del sistema *DIPReader*

5.1 Renderer

5.1.1 Struttura architetturale

Il container *Renderer* è responsabile della presentazione dell'interfaccia utente e dell'interazione con l'utente. Per la gestione quindi dell'interfaccia è stato scelto di strutturare il container per componenti. Ogni componente a sua volta è strutturato da una serie di services che si occupano di gestire la logica di presentazione, e da una serie di componenti di presentazione che si occupano di mostrare i dati all'utente.
Visto poi che le interazioni tra i diversi componenti di presentazione sono limitate, è stato scelto di strutturare il container in modo che ogni componente sia completamente disaccoppiato dagli altri. Gli unici punti di contatto risultano infatti essere il sistema di routing e i canali IPC, che permettono ai componenti di comunicare tra loro e con il backend.
In presenza poi di componenti come per esempio l'esportazione dei file, che richiedono di essere inseriti in più punti del sistema, è stato scelto di strutturare il container in modo che questi componenti siano facilmente integrabili in qualsiasi punto del sistema, senza dover modificare la struttura del container o dei componenti di presentazione grazie a delle semplici Injection di dipendenze.
È poi stato applicato il pattern Smart and Dumb Components, che permette di separare la logica di presentazione pura dalla logica di business che la coordina.

5.2 [Main]

5.2.1 Tecnologia: Node.js

Descrizione

6 Architettura delle componenti

6.1 [Componente 1]

6.1.1 Descrizione

6.1.2 Interfacce

6.1.3 Dipendenze

6.1.4 Tracciamento dei requisiti

6.2 [Componente 2]

6.2.1 Descrizione

6.2.2 Interfacce

6.2.3 Dipendenze

6.2.4 Tracciamento dei requisiti

6.3 Componenti di Ricerca (FrontEnd)

6.3.1 Descrizione

I componenti di ricerca sono responsabili della gestione delle funzionalità di compilazione dei campi di ricerca.
Come già anticipato il pattern applicato è quello di Smart and Dumb Components, per cui i componenti di ricerca sono suddivisi in componenti di pura presentazione e gestiti da i componenti "*Padri*", che si occupano di gestire la logica di questi ultimi nonchè le interazioni con altri servizi come la validazione dei filtri, la gestione dei campi di ricerca dinamici e il routing della sezione dei risultati. Nel contesto della ricerca questo coomponente Padre è SearchPageComponent.
Come si può notare questo componente sfrutta la dependency injection nativa di Angular per risolvere le dipendenze. Inoltre sono iniettate le interfaccie dei servizi necessari per il funzionamento e non gli effettivi servizi, applicando il principio di sostituzione di Liskov.

7 Backend – Search

7.1 Panoramica

Il modulo di ricerca espone al frontend, tramite IPC di Electron, le funzionalità di interrogazione del pacchetto DiP indicizzato. Il punto di ingresso è *SearchIpcAdapter*, che registra su *IpcMain* i canali di comunicazione e delega l'elaborazione ai rispettivi Use Case.
Il modulo supporta quattro modalità di ricerca principali:

- **Ricerca per classe documentale** – ricerca le *DocumentClass* per nome tramite *SearchDocumentalClassUC*.
- **Ricerca per processo** – ricerca i *Process* per UUID tramite *SearchProcessUC*.
- **Ricerca strutturata per metadati** – ricerca i *Document* applicando un albero di filtri logici tramite *SearchDocumentsUC*, costruito a partire da un *SearchRequestDTO* validato con Zod e trasformato in un *SearchDocumentsQuery* tramite *MetadataKeyMapper*.
- **Ricerca semantica** – ricerca i *Document* per similarità vettoriale tramite *SearchSemanticUC*, convertendo la query in un vettore di embedding tramite *IWordEmbedding* e calcolando la *cosine similarity* rispetto ai vettori indicizzati.
- **Ricerca chiavi di metadati** – recupera le chiavi di metadati personalizzati presenti nei documenti del pacchetto (quindi nel Database).

Inoltre è esposto un canale per ottenere lo stato del motore AI, cioè se il modello è stato inizializzato e il numero di documenti indicizzati.

I risultati vengono mappati in DTO di presentazione (*DocumentSearchResult*, *ProcessSearchResult*, *DocumentClassSearchResult*) prima di essere restituiti al frontend, mantenendo il dominio disaccoppiato dal livello di trasporto IPC.

7.2 Architettura del modulo

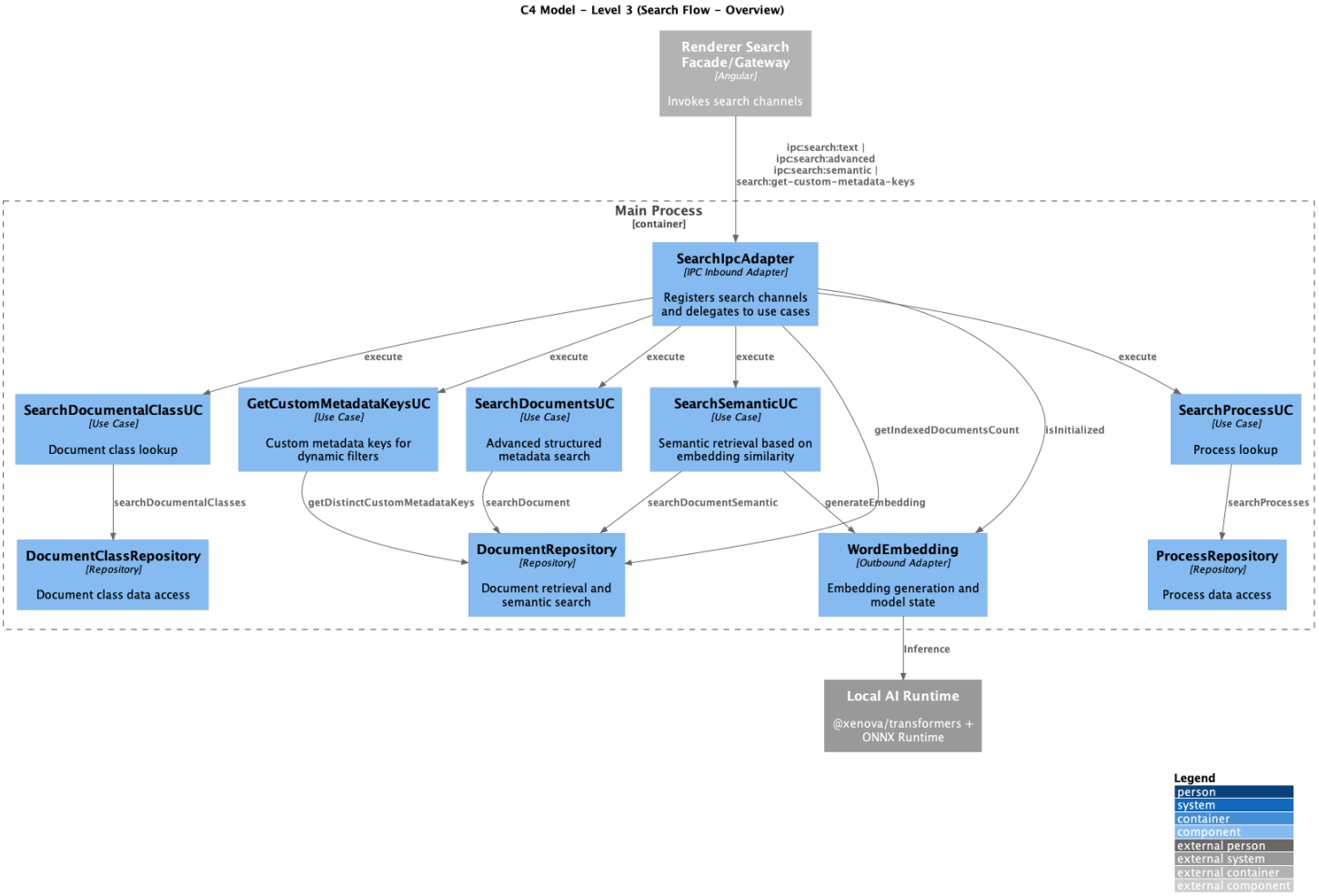

Figura 3: Diagramma C3 – Componenti del modulo di ricerca

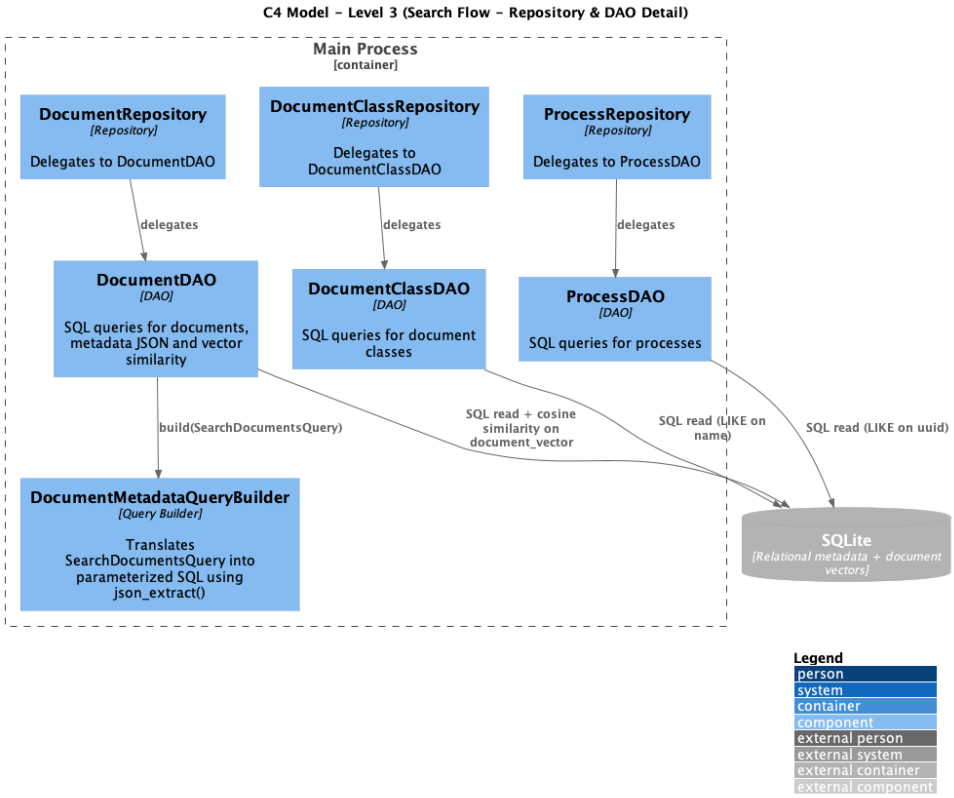

Figura 4: Diagramma C3 – Componenti del modulo di ricerca (Repository e DAO)

7.2.1 SearchIpcAdapter

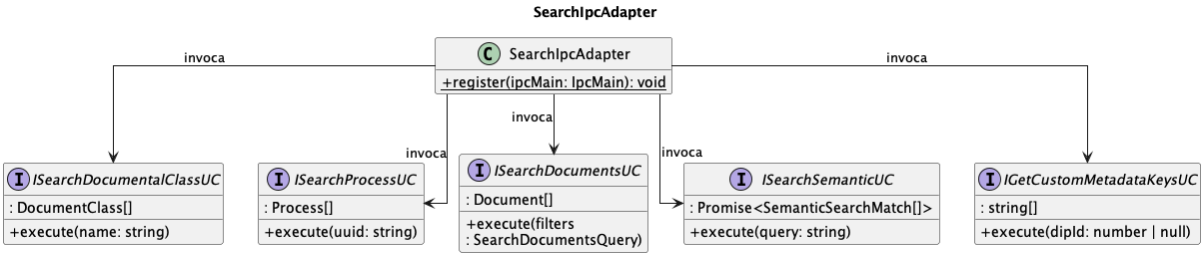

Figura 5: Diagramma delle classi – SearchIpcAdapter

SearchIpcAdapter è l'adapter inbound che registra su *IpcMain* i canali IPC del modulo di ricerca. Risolve le dipendenze tramite il container DI (tsyringe) e instrada le richieste dal renderer process ai rispettivi Use Case, senza contenere logica di business.

Metodo principale *+ register(ipcMain: IpcMain): void* – metodo statico che registra tutti i canali IPC del modulo. Per ciascun canale risolve dal container il corrispondente Use Case o adapter, ne invoca *execute()* e restituisce i risultati mappati in DTO di presentazione.

Canali registrati

- *SEARCH_CLASSES* – ricerca le classi documentali per nome, delegando a *ISearchDocumentalCla* e mappando i risultati tramite *DocumentClassMapper.toSearchResult()*.
- *SEARCH_PROCESSES* – ricerca i processi per UUID, delegando a *ISearchProcessUC* e mappando tramite *ProcessMapper.toSearchResult()*.
- *SEARCH_DOCUMENTS* – ricerca strutturata tramite filtri. Valida il *SearchRequestDTO* con Zod, normalizza i path dei metadati tramite *MetadataKeyMapper.mapGroup()*, costruisce un *SearchDocumentsQuery* e delega a *ISearchDocumentsUC*. I risultati vengono mappati tramite *DocumentMapper.toSearchResult()*.
- *SEARCH_SEMANTIC* – ricerca semantica testuale, delegando a *ISearchSemanticUC* e mappando i risultati (con score di similarità) tramite *DocumentMapper.toSearchResult()*.
- *SEARCH_FULLTEXT* – ricerca full-text polimorfica che, in base al *SearchQueryType* della query (*PROCESS_ID*, *CLASS_NAME*, *FREE*), instrada la richiesta al Use Case appropriato.
- *SEARCH_GET_AI_STATE* – restituisce lo stato del motore AI interrogando *IWordEmbedding.isInit* e *IDocumentRepository.getIndexedDocumentsCount()*, costruendo un oggetto con *status*, *progressPercentage* e *indexedDocuments*.
- *SEARCH_CUSTOM_METADATA_KEYS* – recupera le chiavi di metadati personalizzati delegando a *IGetCustomMetadataKeysUC*.

7.2.2 Use Case di ricerca

I cinque Use Case del modulo implementano ciascuno un'interfaccia dedicata e delegano le operazioni di persistenza ai rispettivi repository, senza contenere logica infrastrutturale.

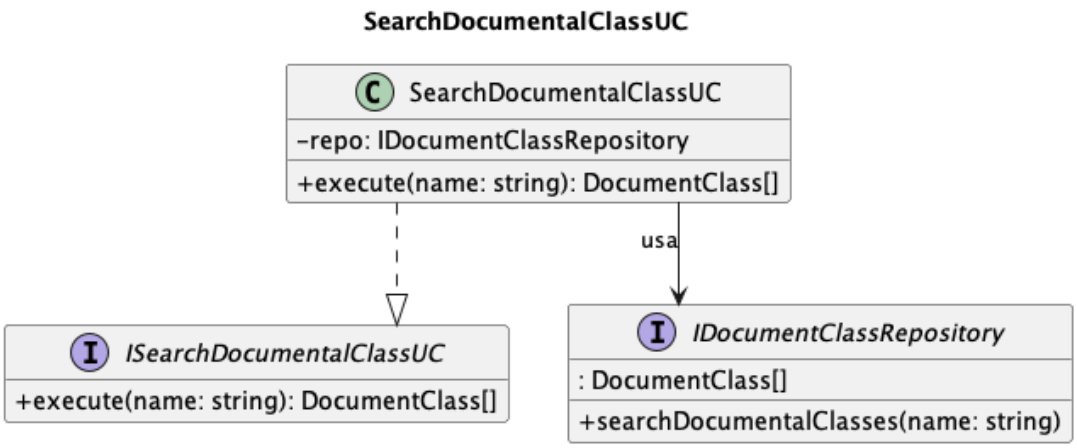

Figura 6: Diagramma delle classi – SearchDocumentalClassUC

SearchDocumentalClassUC Implementa *ISearchDocumentalClassUC*. Il metodo *execute(name: string): DocumentClass[]* invoca

IDocumentClassRepository.searchDocumentalClasses(name) e restituisce le entità corrispondenti, senza logica aggiuntiva.

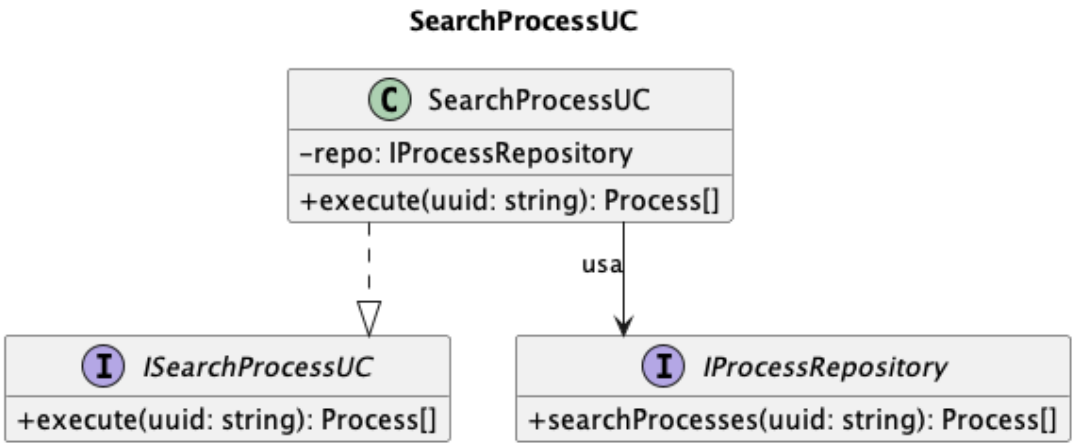

Figura 7: Diagramma delle classi – SearchProcessUC

SearchProcessUC Implementa *ISearchProcessUC*. Il metodo *execute(uuid: string): Process[]* invoca *IProcessRepository.searchProcesses(uuid)* e restituisce le entità corrispondenti.

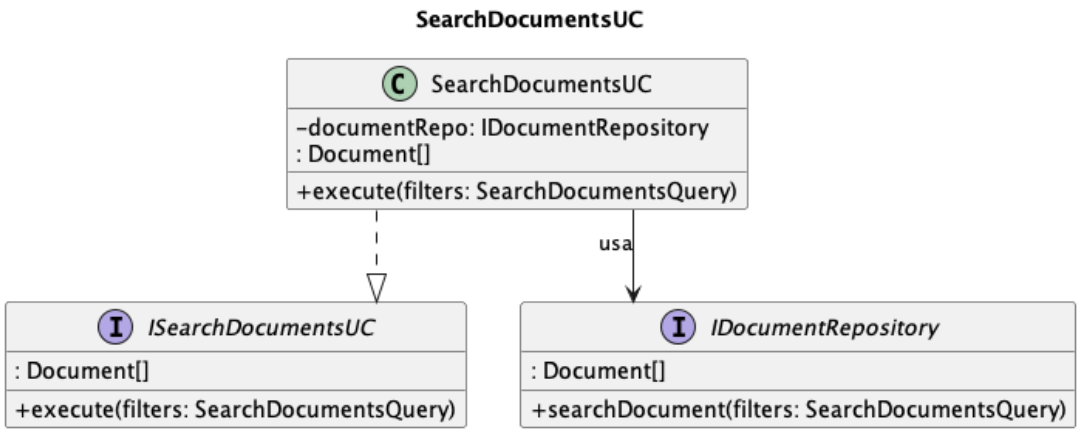

Figura 8: Diagramma delle classi – SearchDocumentsUC

SearchDocumentsUC Implementa *ISearchDocumentsUC*. Riceve un *SearchDocumentsQuery* già costruito dall'adapter. Il metodo *execute(filters: SearchDocumentsQuery): Document[]* invoca *IDocumentRepository.searchDocument(filters)* e restituisce i documenti corrispondenti ai criteri di filtro.

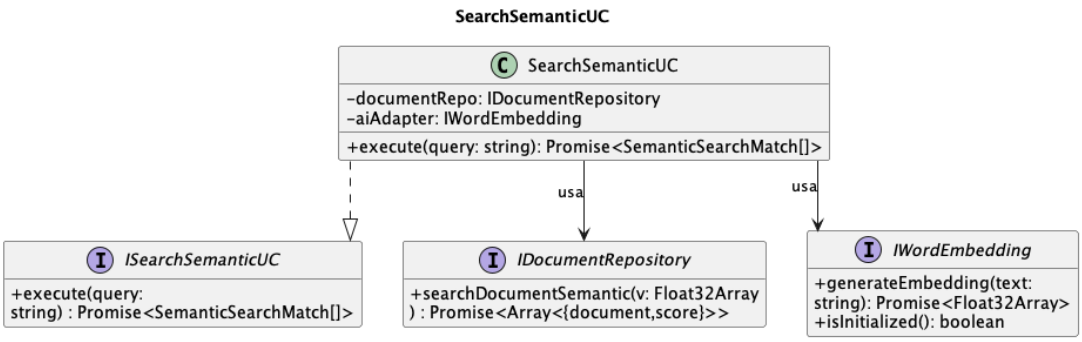

Figura 9: Diagramma delle classi – SearchSemanticUC

SearchSemanticUC Implementa *ISearchSemanticUC*. Coordina *IWordEmbedding* per la generazione del vettore di query e *IDocumentRepository* per la ricerca tra i vettori indicizzati. Il metodo *execute(query: string): Promise<SemanticSearchMatch[]>* converte la query in un vettore tramite *IWordEmbedding.generateEmbedding()*, quindi invoca *IDocumentRepository.searchDocumentSemantic()* e restituisce un array di *SemanticSearchMatch*, ciascuno con il *Document* e il relativo *score* di similarità.

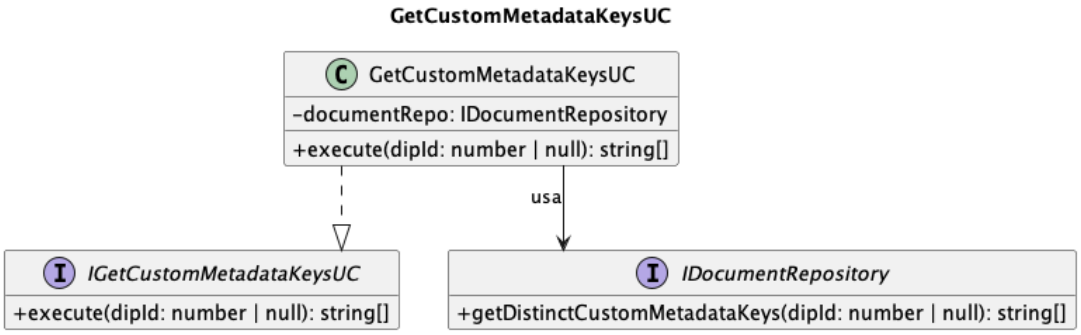

Figura 10: Diagramma delle classi – GetCustomMetadataKeysUC

GetCustomMetadataKeysUC Implementa *IGetCustomMetadataKeysUC*. Il metodo *execute(dipId: number | null): string[]* invoca *IDocumentRepository.getDistinctCustomMetadataKeys(dipId)* e restituisce le chiavi distinte. Se *dipId* è *null*, le chiavi vengono recuperate sull'intero database senza filtrare per pacchetto.

7.2.3 Modelli di dominio per la ricerca

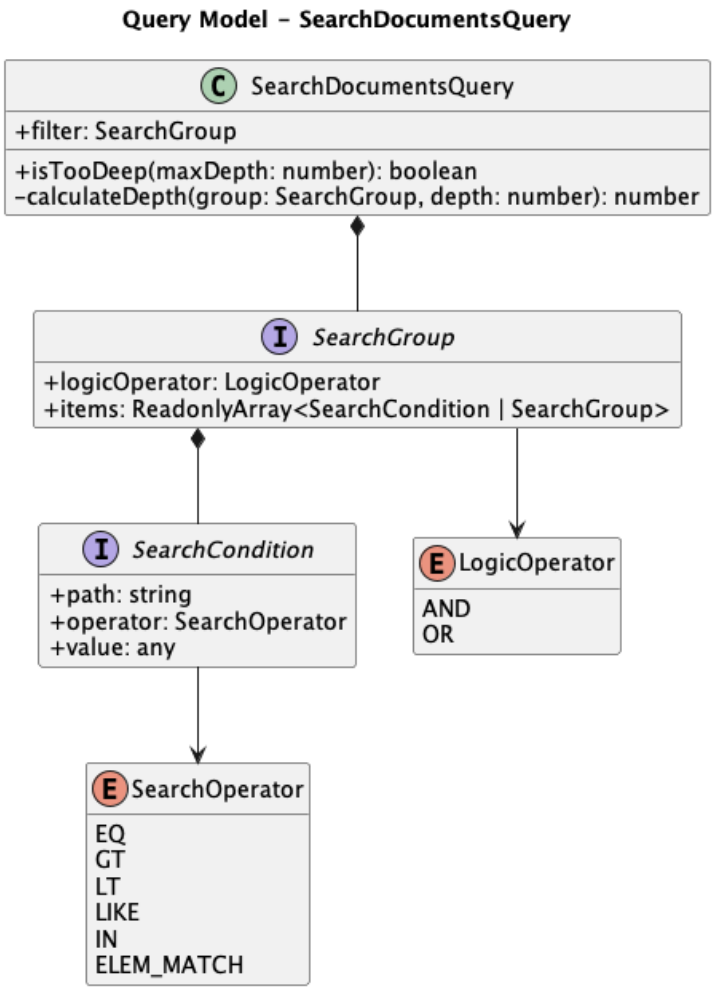

Figura 11: Diagramma delle classi – Query Model: SearchDocumentsQuery, SearchGroup, SearchCondition

SearchDocumentsQuery Oggetto query che incapsula l'albero di filtri logici da applicare alla ricerca dei documenti. Viene costruito dall'adapter a partire dal *SearchRequestDTO* validato e passato allo Use Case e poi al DAO.

- *filter: SearchGroup* – radice dell'albero di filtri.
- *isTooDeep(maxDepth: number): boolean* – calcola ricorsivamente la profondità dell'albero e restituisce *true* se supera *maxDepth* (default 5), per prevenire query eccessivamente annidate.

SearchGroup Interfaccia che rappresenta un nodo dell'albero di filtri. Implementa il ***Composite pattern***: ogni elemento figlio può essere a sua volta un *SearchGroup* (nodo) o una *SearchCondition* (foglia).

- *logicOperator: LogicOperator* – operatore *AND* o *OR* che combina i figli.

- *items: ReadonlyArray<SearchCondition | SearchGroup>* – elementi figli del gruppo.

SearchCondition Interfaccia che rappresenta una foglia dell'albero, ovvero una singola condizione atomica su un campo del documento.

- *path: string* – percorso dot-notation del campo (es. *NomeDelDocumento* o *CustomMetadata.codice*).
- *operator: SearchOperator* – operatore di confronto: *EQ*, *GT*, *LT*, *LIKE*, *IN*, *ELEM_MATCH*.
- *value: any* – valore di confronto. Per *IN* deve essere un array; per *ELEM_MATCH* un *SearchGroup* annidato.

SearchRequestDTO e SearchRequestSchema DTO e schema Zod per la richiesta di ricerca strutturata dal frontend. *SearchRequestSchema* valida ricorsivamente l'albero di filtri, garantendo la correttezza strutturale prima che la richiesta raggiunga il dominio. Il path di ogni *SearchCondition* è validato tramite una regex che consente solo la dot-notation alfanumerica.

SearchQuery Interfaccia per le query di ricerca full-text o semantica provenienti dal frontend.

- *text: string* – testo inserito dall'utente.
- *type: SearchQueryType* – tipo della query (*PROCESS_ID*, *CLASS_NAME*, *FREE*), usato da *SearchIpcAdapter* per l'instradamento nel canale *SEARCH_FULLTEXT*.
- *useSemanticSearch: boolean* – flag per attivare la ricerca semantica.

7.2.4 Mapper di ricerca

I mapper trasformano le entità di dominio nei DTO di presentazione restituiti al frontend. Sono metodi statici collocati nei rispettivi mapper di persistenza per coerenza architetturale.

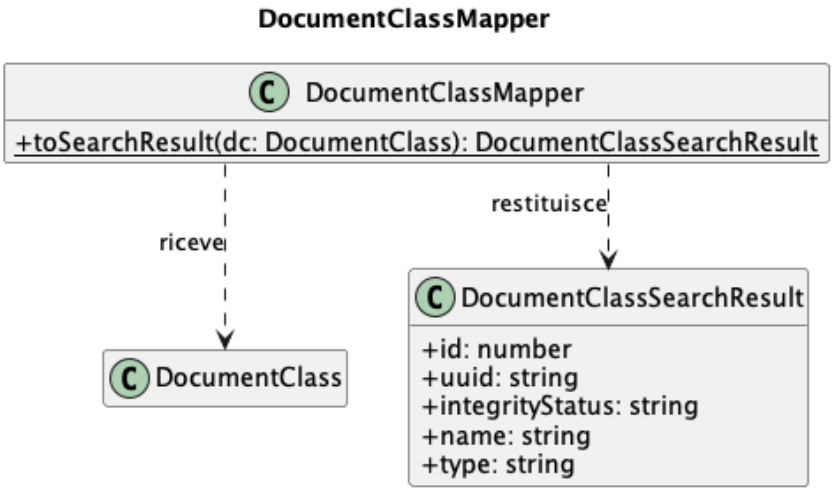

Figura 12: Diagramma delle classi – DocumentClassMapper

DocumentClassMapper.toSearchResult() Trasforma un'entità *DocumentClass* in un *DocumentClassSearchResult*, verificando che l'entità sia già persistita (id valido). Restituisce i campi *id*, *uuid*, *integrityStatus*, *name* e *type* (fisso alle Classi).

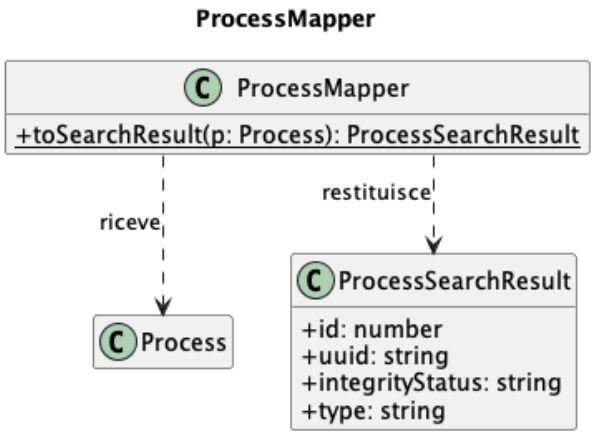

Figura 13: Diagramma delle classi – ProcessMapper

ProcessMapper.toSearchResult() Trasforma un'entità *Process* in un *ProcessSearchResult*, verificando la persistenza. Restituisce i campi *id*, *uuid*, *integrityStatus* e *type* (fisso ai processi).

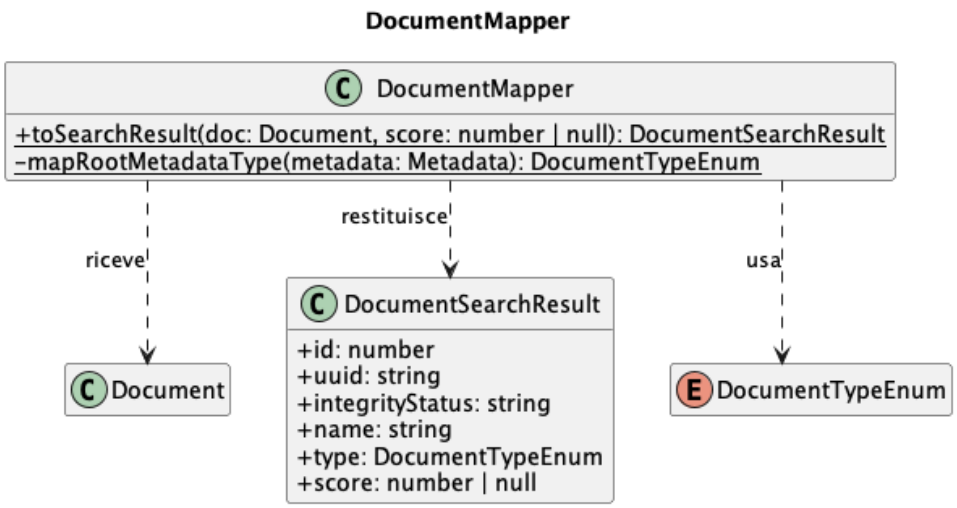

Figura 14: Diagramma delle classi – DocumentMapper

DocumentMapper.toSearchResult() Trasforma un'entità *Document* e uno score opzionale in un *DocumentSearchResult*. Estrae il nome cercando prima il nodo *NomeDelDocumento*, poi *nome*, nell'albero dei metadati. Determina il tipo (*DocumentTypeEnum*) tramite *mapRootMetadataType()*, che normalizza il nome del nodo radice in *UPPER_SNAKE_CASE* e lo confronta con i valori canonici. Restituisce i campi *id*, *uuid*, *integrityStatus*, *name*, *type* e *score*.

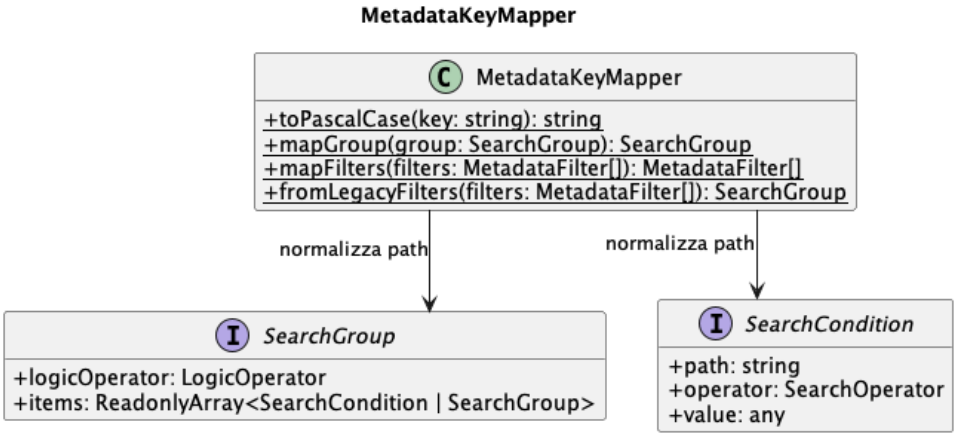

Figura 15: Diagramma delle classi – MetadataKeyMapper

MetadataKeyMapper Utility class che normalizza i path dei metadati dal frontend in PascalCase, garantendo coerenza con la struttura interna del JSON dei metadati.

- *toPascalCase(key: string): string* – converte una chiave in PascalCase, gestendo separatori _ e - e trasformando ogni segmento di path dot-notation separatamente.
- *mapGroup(group: SearchGroup): SearchGroup* – applica ricorsivamente *toPascalCase()* al campo *path* di ogni *SearchCondition* nell'albero, inclusi i valori di condizioni *ELEM_MATCH* con *SearchGroup* annidato.

- *mapFilters(filters: MetadataFilter[]): MetadataFilter[]* – variante per array piatti di filtri chiave-valore; normalizza il campo *key* di ciascun filtro.
- *fromLegacyFilters(filters: MetadataFilter[]): SearchGroup* – metodo di compatibilità che converte un array piatto di filtri legacy in un *SearchGroup* con operatore *AND*, filtrando i valori nulli o vuoti e impostando l'operatore *EQ* per ogni condizione.

7.2.5 Repository e DAO di ricerca

I repository implementano il ***pattern Repository*** delegando le operazioni di persistenza ai rispettivi DAO, che interagiscono direttamente con SQLite tramite *better-sqlite3*.

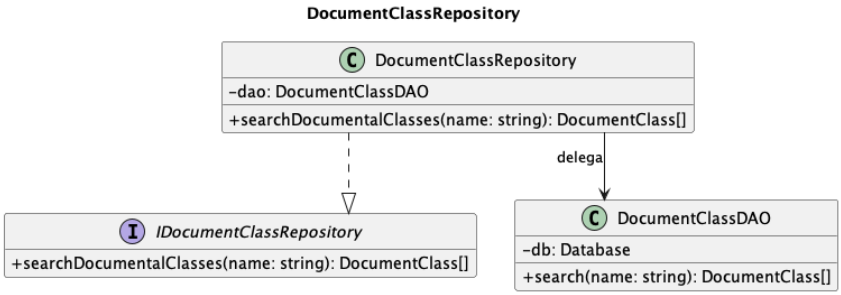

Figura 16: Diagramma delle classi – DocumentClassRepository e DAO

DocumentClassRepository e DocumentClassDAO *DocumentClassRepository* espone *searchDocumentalClasses(name)*, delegando a *DocumentClassDAO.search(name)*, che esegue la query e mappa le righe risultanti in entità *DocumentClass* tramite *DocumentClassMapper.fromPersistence()*.

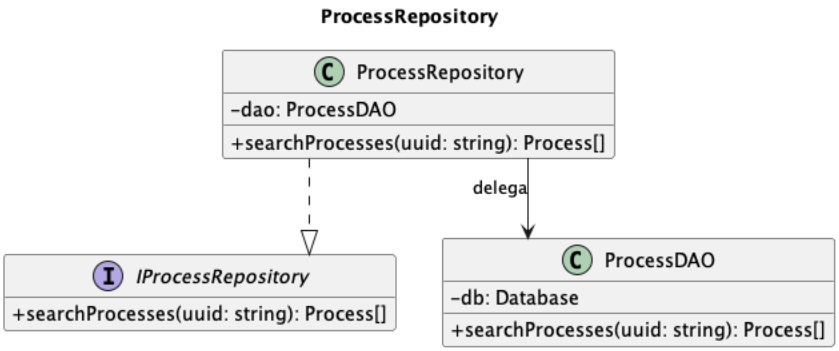

Figura 17: Diagramma delle classi – ProcessRepository e DAO

ProcessRepository e ProcessDAO *ProcessRepository* espone *searchProcesses(uuid)*, delegando a *ProcessDAO.searchProcesses(uuid)*, che esegue la query e mappa le righe risultanti in entità *Process*.

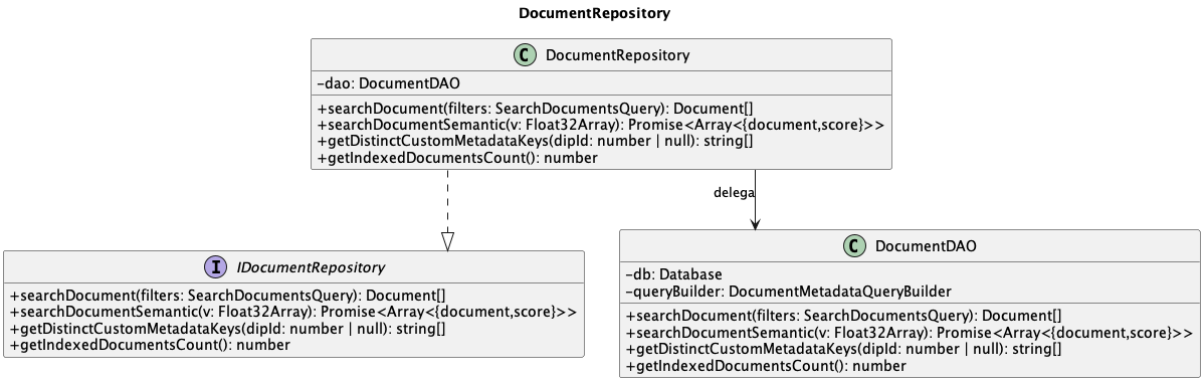

Figura 18: Diagramma delle classi – DocumentRepository e DAO

DocumentRepository e DocumentDAO *DocumentRepository* espone i seguenti metodi di ricerca:

- *searchDocument(filters: SearchDocumentsQuery): Document[]* – delega a *DocumentDAO.searchDocument()*, che utilizza *DocumentMetadataQueryBuilder* per tradurre i filtri in una query ed eseguirla.
- *searchDocumentSemantic(queryVector: Float32Array): Promise<Array<{document, score}> >* – delega a *DocumentDAO.searchDocumentSemantic()*, che esegue la ricerca per similarità vettoriale e restituisce le prime 10 coppie *{document, score}* ordinate per score decrescente.
- *getDistinctCustomMetadataKeys(dipId: number | null): string[]* – esegue la query e restituisce le chiavi di metadati personalizzati distinte, deduplicate e ordinate alfabeticamente.
- *getIndexedDocumentsCount(): number* – esegue la query e restituisce il numero di documenti con vettore indicizzato, o 0 se la tabella non esiste.

7.2.6 DocumentMetadataQueryBuilder

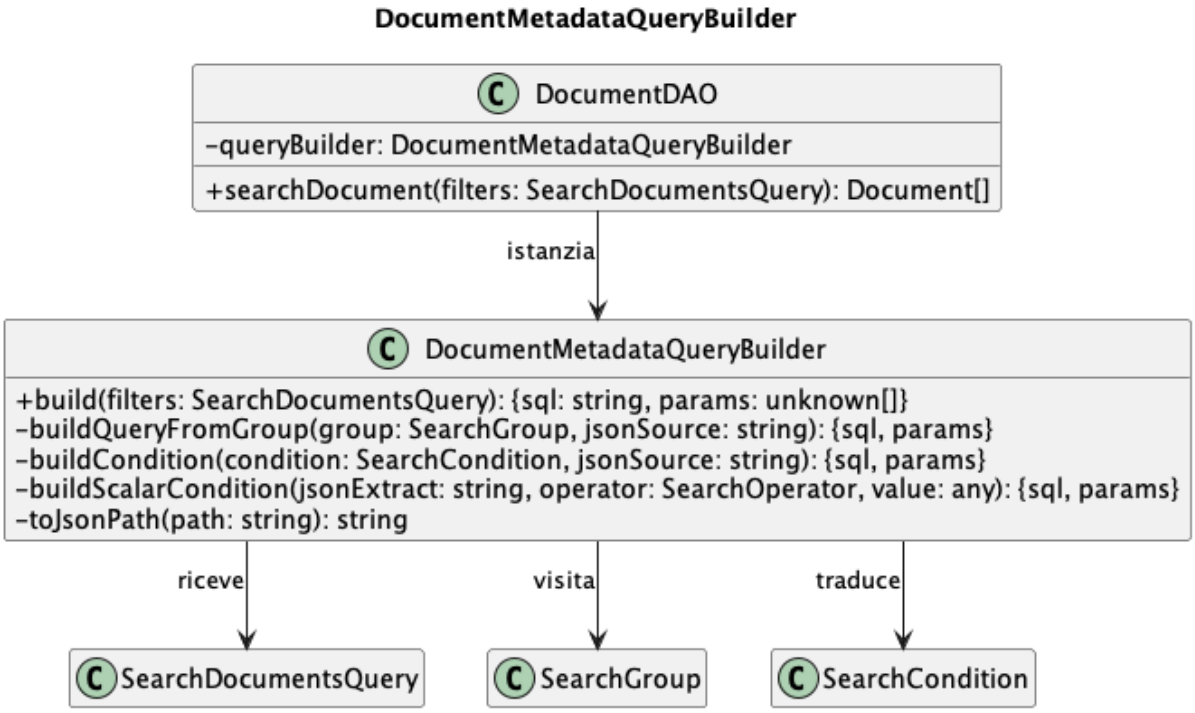

Figura 19: Diagramma delle classi – DocumentMetadataQueryBuilder

DocumentMetadataQueryBuilder traduce un *SearchDocumentsQuery* in una clausola SQL parametrizzata compatibile con SQLite. Viene istanziato direttamente da *DocumentDAO*.

- *build(filters: SearchDocumentsQuery): {sql: string, params: unknown[]}* – punto di ingresso pubblico. Verifica che il filtro non sia vuoto, delega a *buildQueryFromGroup()* sulla radice e restituisce la clausola SQL con i relativi parametri.
- *buildQueryFromGroup(group: SearchGroup, jsonSource: string): {sql, params}* – costruisce ricorsivamente la clausola SQL per un *SearchGroup*, distinguendo nodi (ricorsione) e foglie (*buildCondition()*). Le clausole vengono unite con *AND* o *OR* in base al *logicOperator*.
- *buildCondition(condition: SearchCondition, jsonSource: string): {sql, params}* – traduce una singola *SearchCondition* nella clausola SQL corrispondente, gestendo separatamente l'operatore *ELEM_MATCH* (array annidati) e gli operatori scalari.
- *buildScalarCondition(jsonExtract, operator, value): {sql, params}* – genera la clausola SQL per operatori scalari (*EQ*, *GT*, *LT*, *LIKE*, *IN*), eseguendo le opportune normalizzazioni di tipo e valore.
- *toJsonPath(path: string): string* – converte un path dot-notation nella sintassi JSONPath di SQLite, validando ogni segmento e sollevando un'eccezione in caso di segmenti non validi.

8 Backend – Export e Stampa

8.1 Panoramica

Il modulo di export e stampa gestisce due flussi distinti ma correlati: l'esportazione di file dal pacchetto DiP verso il filesystem locale dell'utente, e la stampa di uno o più file tramite il sistema di stampa nativo del sistema operativo.
L'orchestrazione è gestita da due Use Case dedicati: *ExportFileUC* e *PrintFileUC*. Entrambi ricevono un identificativo numerico del file da elaborare, recuperano la relativa entità tramite *IFileRepository*, risolvono il percorso assoluto all'interno del pacchetto DiP e delegano l'operazione alle rispettive port outbound (*IExportPort* e *IPrintPort*).
Il punto di ingresso lato Electron è *FileViewerIpcAdapter*, che registra i canali IPC su *ipcMain* e funge da ponte tra il processo renderer e i Use Case del backend. Per le operazioni che richiedono interazione con l'utente (scelta del percorso di destinazione, conferma della stampa multipla), l'adapter si avvale delle API native di Electron tramite *dialog*.

8.2 Architettura del modulo

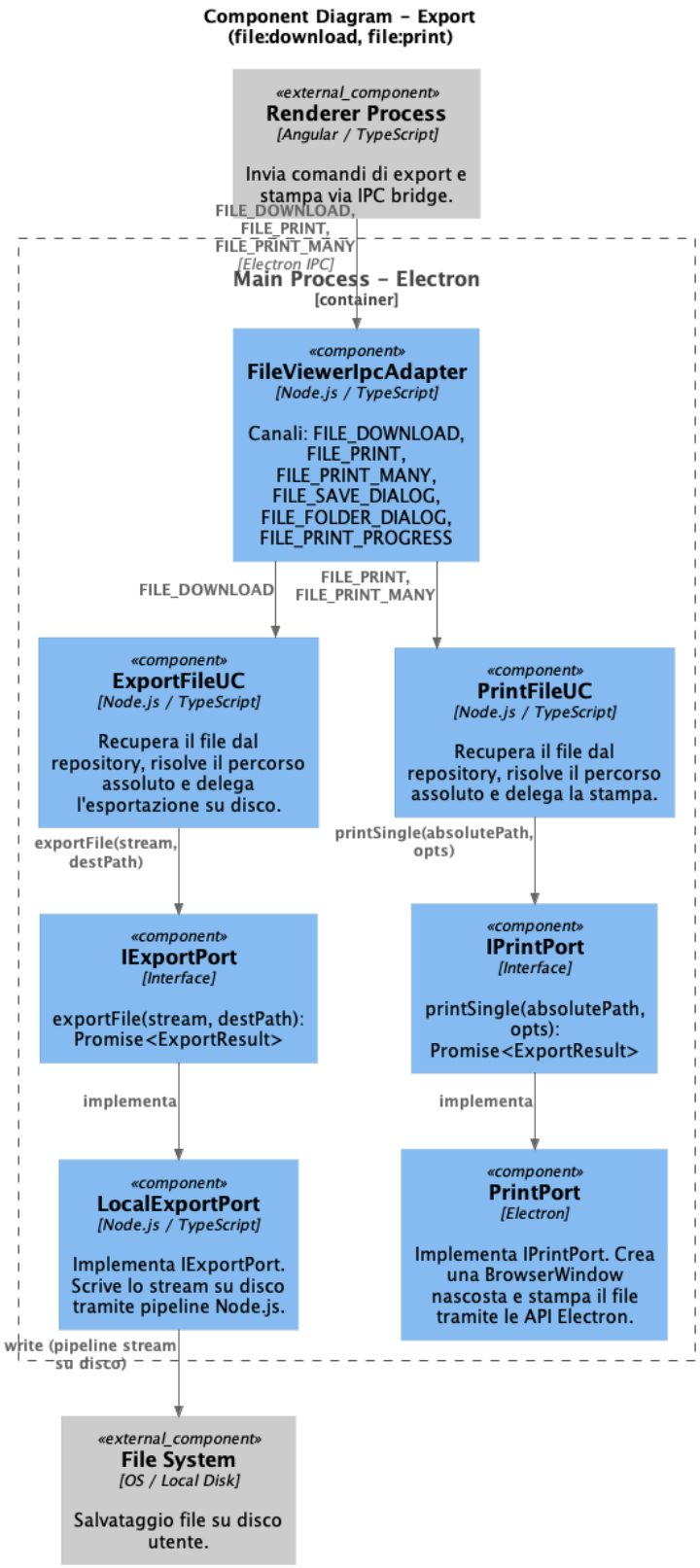

Figura 20: Diagramma C3 – Componenti del modulo di export e stampa

8.3 Componenti

8.3.1 FileViewerIpcAdapter

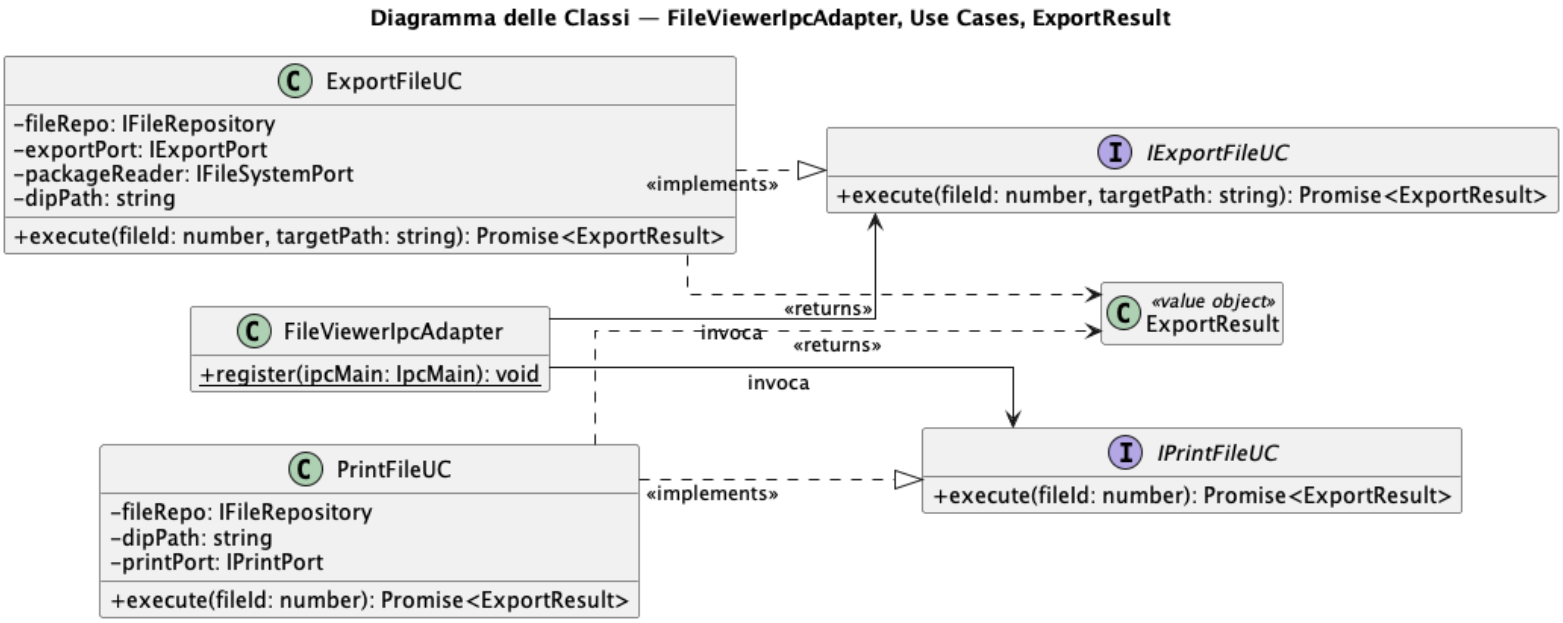

Figura 21: Diagramma delle classi – FileViewerIpcAdapter, Use Case e ExportResult

FileViewerIpcAdapter è l'adapter IPC che registra i canali di comunicazione tra il processo renderer e il processo main di Electron, esponendo le funzionalità di export e stampa al frontend. Risolve le dipendenze *IExportFileUC* e *IPrintFileUC* tramite il container DI *tsyringe*.

Metodo principale *+ register(ipcMain: IpcMain): void* – metodo statico che registra tutti i canali IPC del modulo.

Canali registrati

- *FILE_PRINT* – avvia la stampa di un singolo file dato il suo *fileId*, delegando a *PrintFileUC.execute()*.
- *FILE_PRINT_MANY* – avvia la stampa di più file dati i loro *fileIds*. Prima di procedere mostra una finestra di conferma tramite *dialog.showMessageBox*. Se confermata, itera sui file inviando aggiornamenti di avanzamento tramite il canale *FILE_PRINT_PROGRESS*.
- *FILE_SAVE_DIALOG* – apre la finestra di salvataggio nativa tramite *dialog.showSaveDialog*, restituendo il percorso scelto dall'utente o il flag *canceled*.
- *FILE_DOWNLOAD* – avvia l'esportazione di un file dato *fileId* e *destPath*, delegando a *ExportFileUC.execute()*.

- *FILE_FOLDER_DIALOG* – apre la finestra di selezione directory tramite *dialog.showOpenDialog*, restituendo il percorso della cartella selezionata o il flag *canceled*.

8.3.2 Use Case

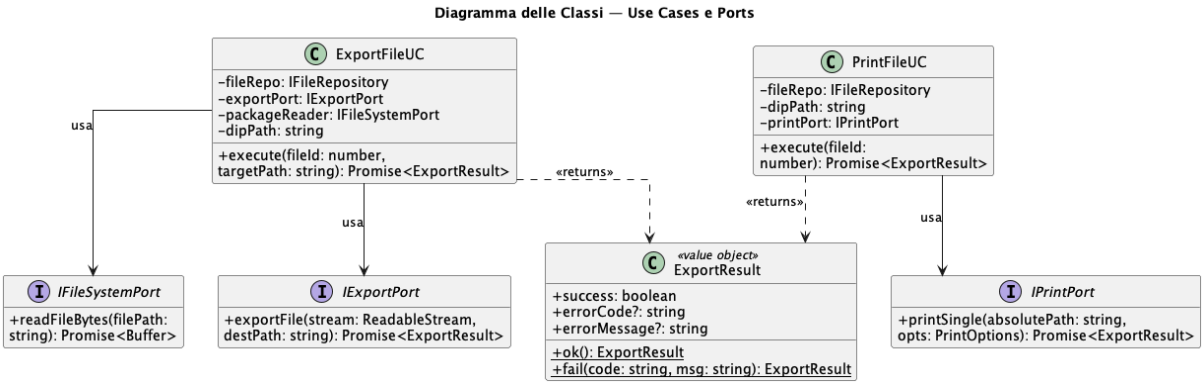

Figura 22: Diagramma delle classi – Use Case e Ports

ExportFileUC Use Case che orchestra il processo di esportazione di un file dal pacchetto DiP verso il filesystem locale. Recupera l'entità *File* tramite *IFileRepository*, ne risolve il percorso assoluto e delega la scrittura su disco a *IExportPort*.
+ execute(fileId: number, targetPath: string): Promise<ExportResult> – avvia l'esportazione del file identificato da *fileId* verso *targetPath*. Se il file non viene trovato nel repository, restituisce un *ExportResult* di fallimento con codice *NOT_FOUND*.

PrintFileUC Use Case che orchestra la stampa di un singolo file. Recupera l'entità *File* tramite *IFileRepository*, ne risolve il percorso assoluto all'interno del pacchetto DiP e delega la stampa a *IPrintPort*.
+ execute(fileId: number): Promise<ExportResult> – avvia la stampa del file identificato da *fileId*. Se il file non viene trovato nel repository, restituisce un *ExportResult* di fallimento con codice *NOT_FOUND*.

8.3.3 ExportResult

Value object che rappresenta l'esito di un'operazione di export o stampa, contenendo le informazioni necessarie a diagnosticare eventuali errori.

- *+ success: boolean* – indica se l'operazione è stata completata con successo.
- *+ errorCode?: string* – codice identificativo dell'errore (es. *NOT_FOUND*, *WRITE_ERROR*), presente solo in caso di fallimento.
- *+ errorMessage?: string* – messaggio descrittivo dell'errore, presente solo in caso di fallimento.
- *+ static ok(): ExportResult* – factory method che restituisce un'istanza di successo.

- *+ static fail(errorCode: string, errorMessage: string): ExportResult* – factory method che restituisce un'istanza di fallimento con il codice e il messaggio forniti.

8.3.4 LocalExportPort e PrintPort

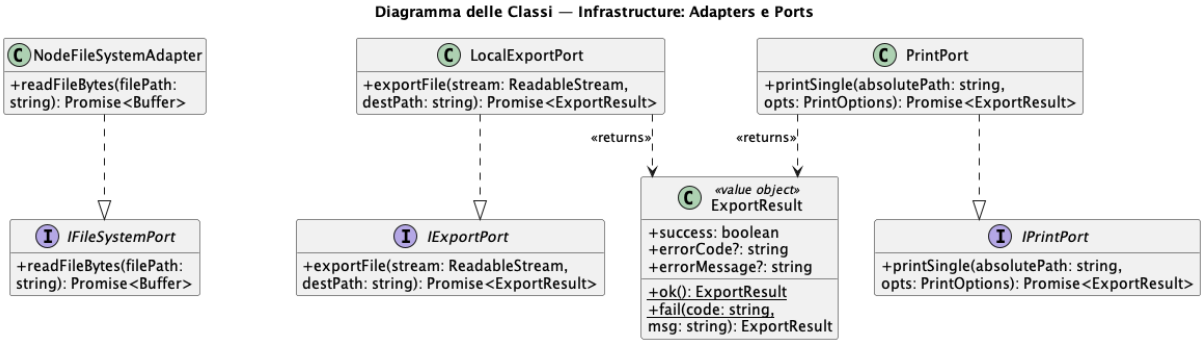

Figura 23: Diagramma delle classi – Infrastructure Layer: adapter e port

LocalExportPort Adapter outbound che implementa *IExportPort*, responsabile della scrittura del contenuto di un file su disco tramite le API di streaming di Node.js. Utilizza *pipeline* di *node:stream/promises* per garantire la corretta gestione degli errori e la chiusura degli stream.
+ exportFile(stream: NodeJS.ReadableStream, destPath: string): Promise<ExportResult> – riceve uno stream leggibile e lo scrive nel percorso di destinazione tramite *createWriteStream*. In caso di errore, restituisce un *ExportResult* di fallimento con codice *WRITE_ERROR*.

PrintPort Adapter outbound che implementa *IPrintPort*, responsabile della stampa di un singolo file tramite le API native di Electron. Crea una *BrowserWindow* nascosta, carica il file tramite il protocollo *file://* e invoca la stampa al termine del rendering.
+ printSingle(absolutePath: string, opts: WebContentsPrintOptions): Promise<ExportResult> – carica il file nel percorso indicato all'interno di una finestra Electron nascosta e avvia la stampa con le opzioni fornite. La finestra viene distrutta al termine dell'operazione, indipendentemente dall'esito. Gestisce sia l'evento *did-finish-load* (stampa avviata correttamente) sia *did-fail-load* (errore di caricamento), restituendo in entrambi i casi un *ExportResult* appropriato.